Lesson 003: One Package, Three LEDs

Common-cathode RGB color mixing with PWM — Arduino Mega 2560

Name: _____ **Date / team:** _____

Board ID: _____ **Observer:** _____

The question

How can three independently controlled LED dice inside one package make colors that appear continuous, and what evidence distinguishes PWM mixing from simply switching three LEDs on or off?

Learning targets. By the end, you can (1) identify common cathode and the three anodes, (2) protect *each* channel with its own resistor, (3) predict and measure RGB PWM duty behavior, (4) explain why duty cycle and PWM frequency are different quantities, and (5) explain the planned C++ composition and RAII lifecycle.

Safety gate — complete before applying USB power

- ☐ Disconnect USB power while building or moving wires.
- ☐ Use a **common-cathode** RGB LED. Confirm its datasheet pinout; package lead order is not universal.
- ☐ Connect the common cathode only to Mega GND.
- ☐ Install **exactly one series resistor per color channel** (three resistors total). Never share one resistor on the common lead.
- ☐ Inspect for a 5 V–GND short and for adjacent legs touching.
- ☐ Power down immediately for heat, odor, flicker unrelated to the program, or a reset loop.

Current budget. Prefer 330Ω on every channel. We design for modest indicator current, not maximum brightness. Do not infer safe current from "it still lights." Before use, check both each pin's current and the aggregate current against the *official ATmega2560 and board documentation*; a per-pin maximum is not a design target, and timer-port/package aggregate limits still apply.

Concept first, wiring second

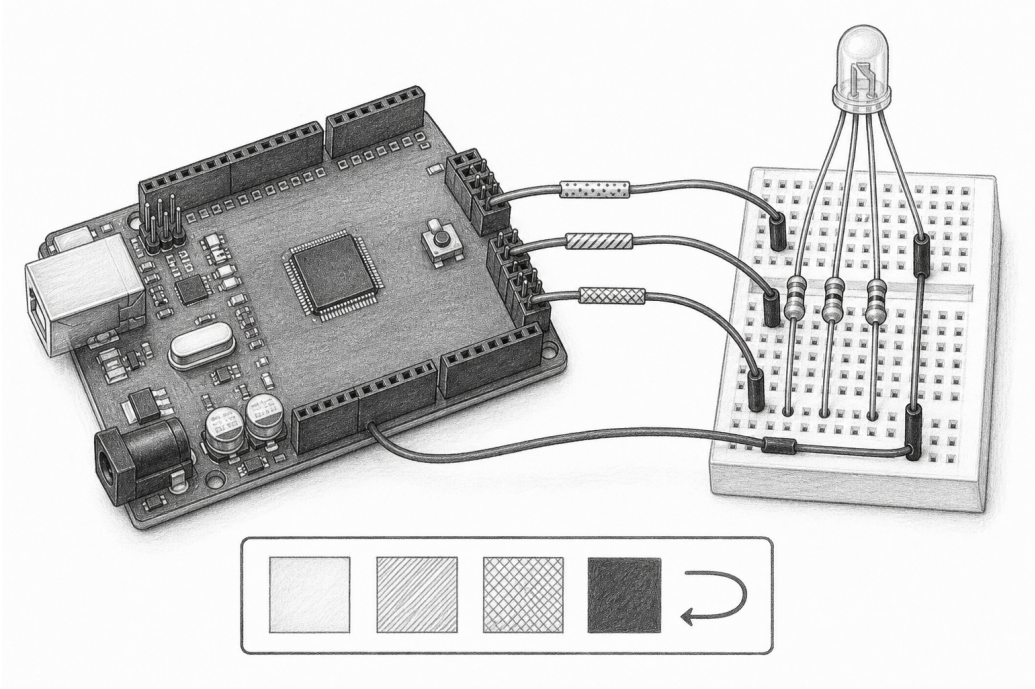
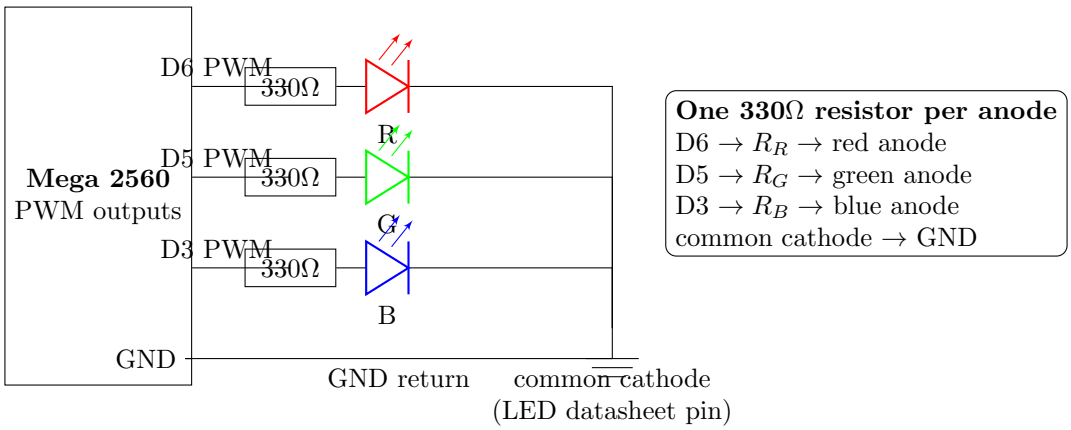


Figure 1 is a conceptual pencil drawing. It communicates the idea of mixed light; it is not an authoritative pin-by-pin wiring diagram. Build only from Figure 2 and the LED manufacturer's pinout.

Exact electrical schematic



Polarity caveat. This circuit is common-cathode, so larger PWM values mean more on-time and usually more light. A common-anode part reverses the logic and cannot be substituted without changing both wiring and software. PWM is rapid digital switching, *not* a true analog voltage. Values 0 and 255 are intended as the off and fully-on endpoints, but exact endpoint behavior is core- and timer-dependent. The high level under load is V_{OH}, not an assumed ideal 5.000 V; it falls as source current rises. Pins 6, 5, and 3 are selected because they support hardware PWM on the Mega 2560; timer configuration by other libraries can change PWM frequency or availability.

Prediction before construction

Assume `on(Color)` maps an 8-bit channel value x to a duty cycle

$$D = \frac{x}{255} \times 100\%.$$

Predict before testing; use words such as dark, dim, medium, bright, or a color name.

Command	R	G	B	predicted duty pattern	predicted appearance
red	255	0	0	R 100%, others 0%	
green	0	255	0	G 100%, others 0%	
blue	0	0	255	B 100%, others 0%	
orange	____	____	0	_____	_____

Prediction claim: If red and green PWM are both active, then I predict _____, because _____.

Setup and controlled state sequence

Equipment: Mega 2560, verified common-cathode RGB LED, three equal 330Ω resistors, breadboard, jumpers, USB cable, and optionally a multimeter or phone slow-motion camera.

1. **State 0 — powerless inspection.** USB disconnected. Identify pins from the LED datasheet. Build Figure 2. Record resistor markings or measured values.
2. **State 1 — isolation test.** Keep two anode jumpers disconnected. Connect one channel at a time and run only its primary color. Repeat R, G, B.
3. **State 2 — composed output.** Connect all three channels. Run the supplied Lesson 003 sequence: red, green, blue, orange, 250 ms each.
4. **State 3 — controlled comparison.** Make this explicit temporary sketch change: add `const auto halfRed = adk::color::Rgb(128, 0, 0);` beside the other colors, then add `led.on (halfRed); delay(intervalMs);` immediately after full red. This uses the supplied library's public constructor and changes no library implementation. Keep pin, resistor, viewing position, room light, interval, and observation method fixed.
5. **State 4 — frequency evidence (instrument optional).** With an oscilloscope or logic analyzer, measure period T on D6 at red value 128 and calculate $f = 1/T$. Then measure at 255. Duty should change; frequency may be unmeasurable at the fully-on endpoint because there are no transitions. Without such an instrument, record "not measured" rather than treating camera bands or appearance as a frequency measurement.
6. **State 5 — power-down.** Disconnect USB before rewiring. This firmware does not yet supply a component-level lifecycle experiment.

Record repeated evidence

Do not use one glance as proof. Complete at least three cycles. "Observed" should describe what happened, not what was expected.

Trial	command	observed color	flicker/transition	unexpected behavior	instrument evidence
1	R/G/B/orange				
2	R/G/B/orange				
3	R/G/B/orange				

D6 command	measured T	calculated f	measured duty	interpretation
128				
255				

Controlled comparison	PWM value	predicted duty	measured or observed result	constants held
Full red	255	100%		
Half red	128	_____		

Required calculation. Show the half-red duty calculation:

$$D = \frac{\text{\_\_\_\_\_}}{255} \times 100\% = \text{\_\_\_\_\_}\%.$$

If a multimeter reports average pin voltage, a rough ideal model is $V_{\text{avg}} \approx D \times 5.0\,\text{V}$. Predict half-red: _____ V.

Explain why LED brightness is not necessarily half:

Current-limiting calculation

For each die, estimate

$$I_{\text{on}} = \frac{V_{OH} - V_f}{R}.$$

Use the LED datasheet V_f, not a color-name guess, and use a measured loaded V_{OH} or a conservative value justified from the MCU datasheet. Example only: if measured $V_{OH} = 4.8$ V, red $V_f = 2.0$ V, and $R = 330\,\Omega$, then $I_{\text{on}} = (4.8 - 2.0)/330 = 8.5$ mA. At 50% duty, the simplified channel average is about 4.2 mA. Repeat for the actual parts, then sum the worst simultaneous channel currents and compare that aggregate with every applicable documented limit:

channel	measured R	datasheet V_f	calculated I_{on}	at tested duty, I_{avg}
red				
green				
blue				

$$I_{\text{aggregate,worst}} = I_R + I_G + I_B = \_\_\_\_\_ \text{ mA}.$$

The program and the object model

```
#include <adk.h>

adk::led::Rgb led(6, 5, 3);

const auto red = adk::color::red();
const auto green = adk::color::green();
const auto blue = adk::color::blue();
const auto orange = adk::color::orange();

void setup()
{
    adk::initialize();
}

void loop()
{
    constexpr auto intervalMs = 250;

    led.on (red); delay(intervalMs);
    led.on (green); delay(intervalMs);
    led.on (blue); delay(intervalMs);
    led.on (orange); delay(intervalMs);
}
```

Composition. An RGB LED is not a special kind of one-pin LED. It *has three* independently driven channels, so composition models the circuit: an `Rgb` object owns or contains red, green, and blue output state. A `Color` value describes desired intensities; it does not own pins.

Planned lifecycle, not a claim about this firmware. The current example constructs a global `Rgb` and calls the library-level `adk::initialize()`; it does not demonstrate per-component `initialize()`, `shutdown()`, pin claiming, or destructor-driven hardware cleanup. The target design is for explicit component `initialize()` to acquire/configure transactionally, for idempotent `shutdown() noexcept` to return outputs to their documented safe state, and for the destructor to invoke that cleanup. In an exception-enabled host, ordinary stack unwinding would then invoke destructors. These are interface requirements to implement and test, not evidence supplied by this circuit. Copying a future pin-owning object should be prohibited so ownership cannot be duplicated.

Why code lives out of line. Headers should expose the small contract while implementation belongs in `.cpp` files. This reduces dependencies and rebuild cost, avoids accidental code duplication, and lets the linker discard unused features. Templates or genuinely compile-time operations are exceptions, not the default.

Interpretation caveats

- Equal numeric R, G, and B values do not guarantee equal perceived brightness. LED efficiency, forward voltage, package optics, and human vision differ.
- A camera may show bands because its shutter samples PWM. That is evidence of switching and sampling, not necessarily visible flicker.
- `delay()` makes this demonstration simple but blocks input handling. Later components should use time-based updates so buttons can remain responsive.
- Apparent orange is additive light mixing. It is not evidence that photons themselves became "orange paint."

Troubleshooting by symptom

Symptom	likely cause	discriminating test (power off before rewiring)
Nothing lights	wrong common pin, common-anode part, no ground, or no upload	Verify datasheet and continuity; run one isolated channel.
One color missing	wrong LED lead, open jumper/resistor, damaged die, wrong pin	Swap only the signal source between a known-good channel and failed channel.
Colors are permuted	anodes assigned to different physical dice	Run isolation State 1 and label each lead from evidence.
Very bright / board resets	missing or bypassed resistor; short circuit	Disconnect immediately; count three distinct series resistors and inspect rails.
Only primary colors work	mixed-color values or mapping is wrong	Command two known nonzero channels and observe each separately first.
Unexpected inversion	common-anode device or inverted driver assumption	Check common lead polarity with datasheet; do not "fix" by guessing.
Camera bands	camera shutter interacting with PWM	Compare direct observation and two frame-rate settings.

Assessment

1. Why is one resistor in the common cathode lead not equivalent to three resistors?

2. With PWM value 64, calculate ideal duty cycle.

3. Which single variable changed in State 3? Name two controlled variables.

4. Explain why `Rgb` should compose channels rather than inherit from one channel.

5. Which lifecycle behavior is present in today's sketch, and which behavior is only a target requirement?

6. A color is wrong. Propose a test that distinguishes software channel mapping from a failed LED die.

CER exit ticket

Claim. State what the repeated trials show about PWM color mixing.

Evidence. Cite at least two specific observations or measurements, including the controlled comparison.

Reasoning. Connect duty cycle, average channel output, additive mixing, and your evidence. Include one limitation or alternative explanation.

Extension without changing the safety envelope

Replace blocking delays with an elapsed-time state machine, then use a debounced button to select the displayed color. Keep the same three resistors and pin mapping. Before coding, draw the states and transitions; afterward, collect three repeated button-to-color response trials. A stronger extension calibrates channel values to make a visually neutral white and records the unequal values as evidence of the system's physical response.